REVISION CHEAT SHEET:

SQL Server Window Ranking Functions

COMPLETE INTERVIEW-READY NOTES - ROW_NUMBER - RANK - DENSE_RANK - NTILE - PARTITION BY

TOPIC & SUBTOPIC INDEX

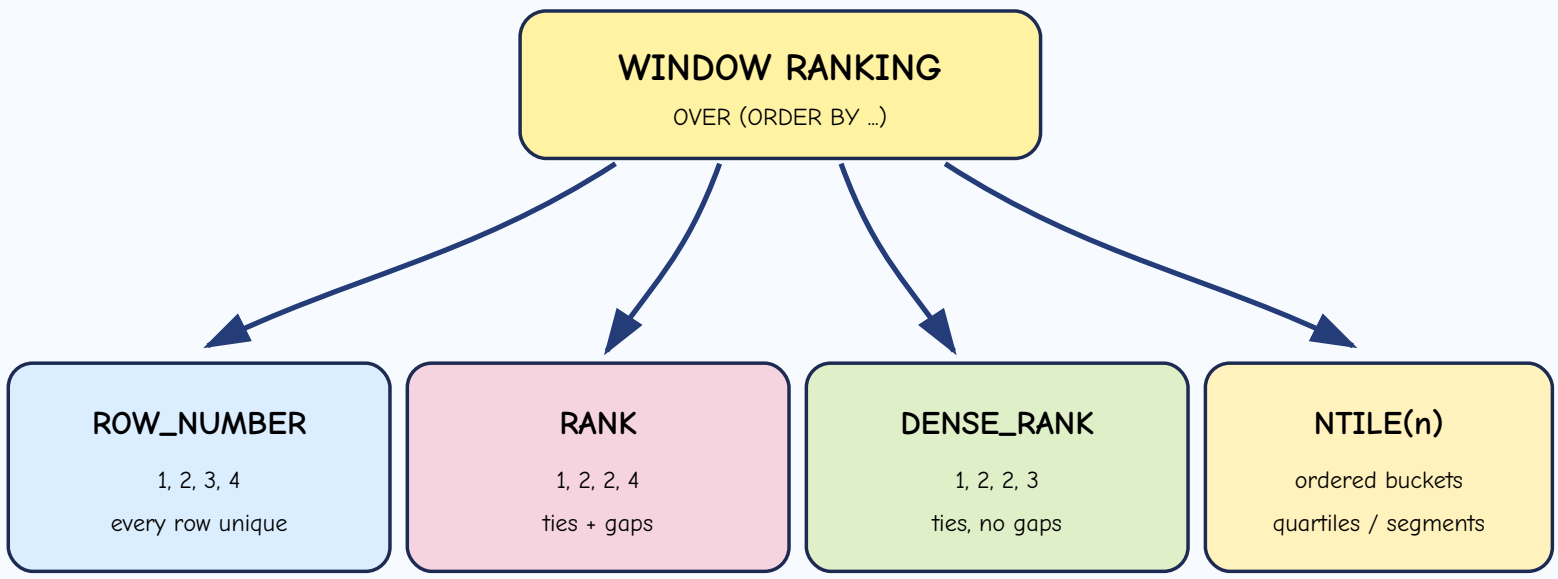
Compact study order: Ranking Functions Overview -> ROW_NUMBER -> ROW_NUMBER Syntax & Rules -> PARTITION BY -> ROW_NUMBER + CTE Top-N Pattern -> RANK -> DENSE_RANK. Related details continue in the same flow without unnecessary blank pages.

- Ranking Functions Overview
- ROW_NUMBER
- ROW_NUMBER Syntax & Rules
- PARTITION BY
- ROW_NUMBER + CTE Top-N Pattern
- RANK
- DENSE_RANK
- ROW_NUMBER vs RANK vs DENSE_RANK
- When to Use Each Ranking Function
- NTILE
- Ranking Functions Cheat Sheet

Ranking Functions Commonly Used with CTEs

3D DIAGRAM

Window Ranking Function Family



All four are ranking functions, but they handle ties or grouping differently.

Ranking functions assign a number or ranking value to rows.

They use the:

```sql
OVER(...)
```

clause.

The most important three are:

EXAMPLE / FLOW

```
ROW_NUMBER
RANK
DENSE_RANK
```

ROW_NUMBER()

Simple Definition

`ROW_NUMBER()` gives every row a unique sequential number.

Example:

EXAMPLE / FLOW

```
1
2
3
4
5
```

Even when two values are equal, they still receive different row numbers.

Syntax

SQL

```
ROW_NUMBER()
OVER
(
    ORDER BY column
)
```

Example:

SQL

```
SELECT
    EmployeeId,
    EmployeeName,
    Salary,

    ROW_NUMBER()
    OVER
    (
        ORDER BY Salary DESC
    ) AS RowNumber

FROM Employees;
```

Suppose salaries are:

Employee	Salary
John	100000
Priya	90000
David	90000
Sarah	80000

Result may be:

Employee	Salary	RowNumber
John	100000	1
Priya	90000	2
David	90000	3
Sarah	80000	4

Notice:

EXAMPLE / FLOW

```
Priya = 90000
David = 90000
```

but they still receive different numbers.

Important ROW_NUMBER Rule

`ROW_NUMBER()` requires ordering inside `OVER`.

```sql
ROW_NUMBER()
OVER
(
    ORDER BY Salary DESC
)
```

The ordering determines how numbers are assigned.

If the ordering columns are not unique, the exact assignment between tied rows can vary.

For deterministic results, add a unique tie-breaker:

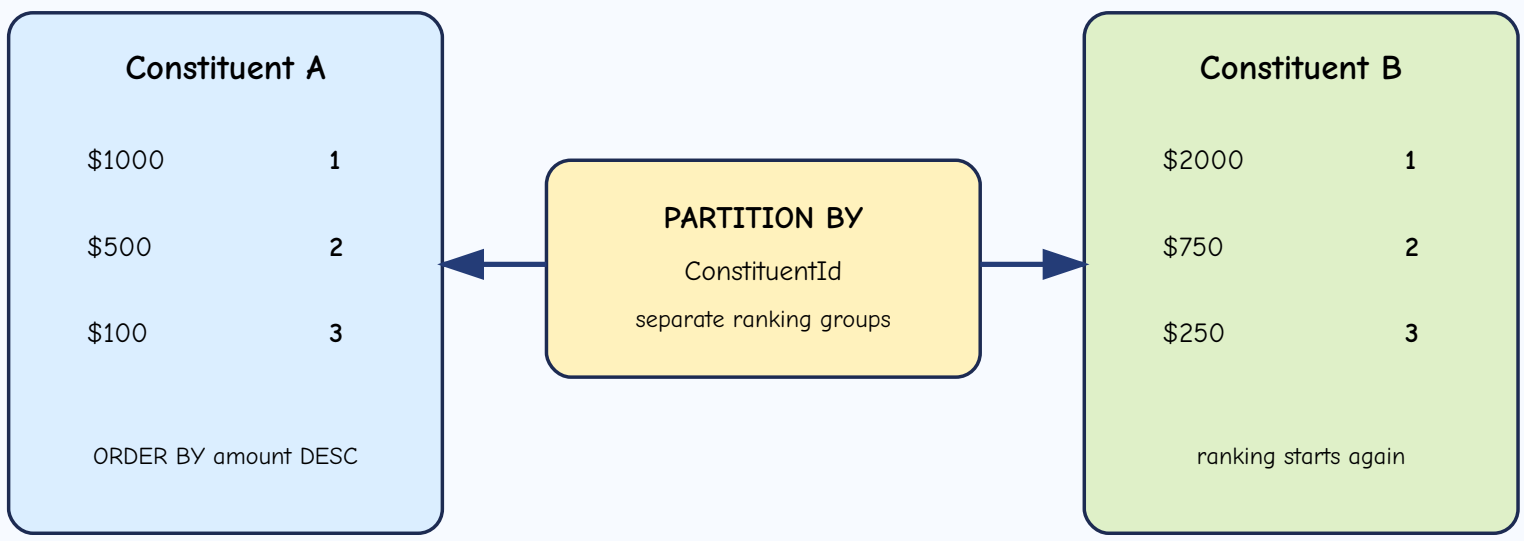
```sql
ROW_NUMBER()
OVER
(
    ORDER BY Salary DESC,
             EmployeeId
)
```

Microsoft notes that `ROW_NUMBER()` is nondeterministic when the partition/order values don't uniquely establish row order.

PARTITION BY with ROW_NUMBER

3D DIAGRAM

PARTITION BY Restarts the Ranking



PARTITION BY divides rows into independent groups; ORDER BY controls the ranking inside each group.

`PARTITION BY` divides rows into separate groups before numbering.

Example:

```sql
SELECT
    EmployeeId,
    DepartmentId,
    Salary,

    ROW_NUMBER()
    OVER
    (
        PARTITION BY DepartmentId
        ORDER BY Salary DESC
```

```
    ) AS RowNumber
FROM Employees;
```

Conceptually:

EXAMPLE / FLOW

```
IT Department
--------------
1
2
3

HR Department
--------------
1
2
3

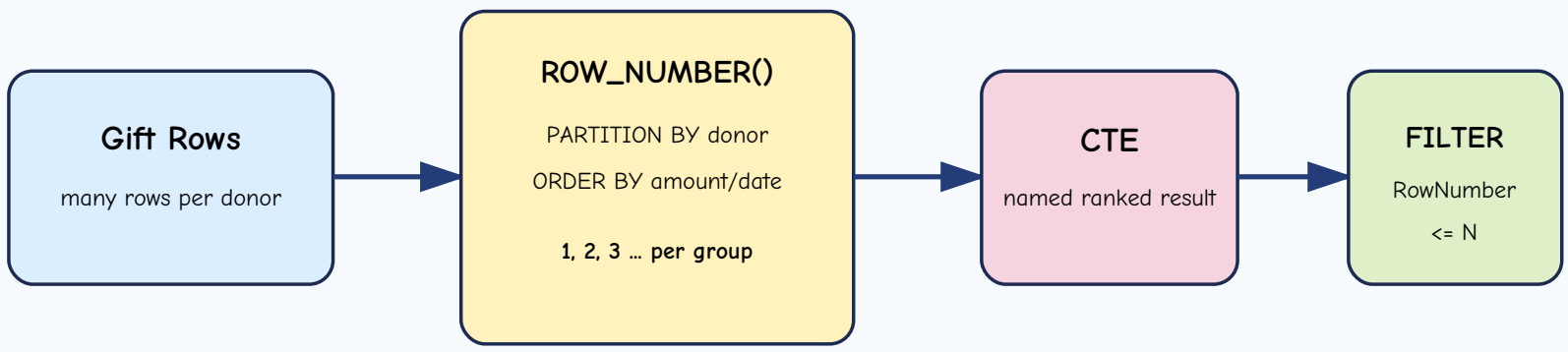
Finance
--------------
1
2
3
```

The numbering restarts inside every department.

ROW_NUMBER with a CTE - Top Employee Per Department

3D DIAGRAM

Top-N per Group with ROW_NUMBER + CTE



Classic uses: latest row per entity, top gifts per constituent, de-duplication, pagination.

Ranking is calculated first; the outer query filters the ranking result cleanly.

This is one of the most common CTE patterns.

SQL

```
WITH RankedEmployees AS
(
    SELECT
        EmployeeId,
        EmployeeName,
        DepartmentId,
```

```
        Salary,

        ROW_NUMBER()
        OVER
        (
            PARTITION BY DepartmentId
            ORDER BY Salary DESC,
                     EmployeeId
        ) AS RowNumber

    FROM Employees
)

SELECT
    EmployeeId,
    EmployeeName,
    DepartmentId,
    Salary
FROM RankedEmployees
WHERE RowNumber = 1;
```

Meaning:

EXAMPLE / FLOW

```
Step 1
→ Rank employees inside each department

Step 2
→ Keep row number 1

Result
→ Highest-ranked employee in each department
```

Why CTE Is Useful with ROW_NUMBER

You generally cannot write:

SQL

```
SELECT
    EmployeeId,
    ROW_NUMBER() OVER (ORDER BY Salary DESC) AS RowNumber
FROM Employees
WHERE RowNumber = 1;
```

because the alias produced by the window calculation isn't available to that `WHERE` clause.

A CTE creates a logical intermediate result:

EXAMPLE / FLOW

```
Employees
    ↓
Calculate ROW_NUMBER
    ↓
CTE
    ↓
Filter RowNumber
```

RANK() Simple Definition

`RANK()` gives tied values the same rank.

But after a tie:

! It leaves gaps.

Example:

EXAMPLE / FLOW

```
1
2
2
4
5
```

RANK Example

SQL

```
SELECT
    EmployeeName,
    Salary,

    RANK()
    OVER
    (
        ORDER BY Salary DESC
    ) AS SalaryRank

FROM Employees;
```

Suppose:

EXAMPLE / FLOW

```
Salary
------
100000
90000
90000
80000
```

Result:

EXAMPLE / FLOW

```
100000 → Rank 1
90000  → Rank 2
90000  → Rank 2
80000  → Rank 4
```

Rank 3 is skipped because two rows occupied rank 2.

Microsoft documents this tie-and-gap behavior for **RANK**.

DENSE_RANK()

Simple Definition

`DENSE_RANK()` also gives tied values the same rank.

Unlike **RANK**, it does **not leave gaps**.

Example:

EXAMPLE / FLOW

```
1
2
2
3
4
```

DENSE_RANK Example

```sql
SELECT
    EmployeeName,
    Salary,

    DENSE_RANK()
    OVER
    (
        ORDER BY Salary DESC
    ) AS SalaryRank

FROM Employees;
```

Result:

```
100000 → 1
90000  → 2
90000  → 2
80000  → 3
```

ROW_NUMBER vs RANK vs DENSE_RANK

Suppose values are:

```
100
90
90
80
```

Result:

Value	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
90	2	2	2
90	3	2	2
80	4	4	3

Memory:

```
ROW_NUMBER
→ Every row gets a different number

RANK
→ Ties share rank, gaps appear

DENSE_RANK
→ Ties share rank, no gaps
```

When to Use ROW_NUMBER

Use when every row should have a unique sequence.

Common examples:

```
Top 1 row per group

Latest record per customer
```

```
Remove duplicates

Pagination

Sequential display numbering
```

When to Use RANK

Use when the ranking itself matters and tied positions should consume positions.

Example:

EXAMPLE / FLOW

```
Competition ranking
```

If two people finish second:

EXAMPLE / FLOW

```
1st
2nd
2nd
4th
```

When to Use DENSE_RANK

Use when ties should share rank but ranks should remain continuous.

Example:

EXAMPLE / FLOW

```
Salary bands
Score ranking
Distinct value ranking
```

NTILE()

`NTILE()` divides ordered rows into approximately equal groups.

Example:

SQL

```sql
SELECT
    EmployeeName,
    Salary,

    NTILE(4)
    OVER
    (
        ORDER BY Salary DESC
    ) AS SalaryQuartile

FROM Employees;
```

Here:

EXAMPLE / FLOW

```
NTILE(4)
```

divides rows into four groups.

Common uses:

EXAMPLE / FLOW

```
Quartiles

Percentile-style groups

Customer segmentation
```

Ranking Functions Cheat Sheet

Function	Ties	Gaps
`ROW_NUMBER()`	Different number	No
`RANK()`	Same rank	Yes
`DENSE_RANK()`	Same rank	No
`NTILE(n)`	Groups rows	Not applicable